

OPTICAL CHARACTER RECOGNITION AND TEXT TO SPEECH GENERATION

A Project Report Submitted to the Madurai Kamaraj University in Partial
Fulfillment for the Award of the Degree

BACHELOR OF COMPUTER APPLICATIONS

(2022 - 2025)

Submitted by

A. NIRMAL ANAND

22BCA152

Under the Guidance of

Dr. S. SHAIK PARVEEN, M.Sc., B.Ed., M.Phil., Ph.D.,



Since 1881

THE AMERICAN COLLEGE

An Autonomous Institution affiliated to Madurai Kamaraj University

(Re-accredited (3rd cycle) by NAAC with Grade “A+” CGPA-3.47 on a 4-points scale)

Madurai - 625002



Since 1881

THE AMERICAN COLLEGE DEPARTMENT OF COMPUTER APPLICATIONS

An Autonomous Institution affiliated to Madurai Kamaraj University
(Re-accredited (3rd cycle) by NAAC with Grade “A+” CGPA-3.47 on a 4-points scale)
Madurai – 625002

CERTIFICATE

This is to certify that this Project titled **OPTICAL CHARACTER RECOGNITION AND TEXT TO SPEECH GENERATION** is a Bonafide work done by **A. Nirmal Anand (22BCA152)** in Partial Fulfillment for the award of the degree of **Bachelor of Computer Applications** of **The American College, Madurai** for the year 2022-2025.

Dr. S. Shaik Parveen, M.Sc., B.Ed., M.Phil., Ph.D.,
Internal Guide

Mrs. T. Suganya, M.Sc., M.Phil., SLET, (Ph.D.,)
Head of the Department

Submitted for the Viva – Voce held on _____

Internal Examiner

External Examiner

DECLARATION

I hereby declare that this project was carried out by me under the guidance of **Dr. S. Shaik Parveen, M.Sc., B.Ed., M.Phil., Ph.D.**, Department of Computer Applications, The American College, Madurai.

I also declare that this project report is the result of my own effort and that it has not been copied from any one and has not been submitted by anybody in anywhere.

Place : Madurai

Date :

NIRMAL ANAND A

(22BCA152)

ACKNOWLEDGEMENT

I am pleased to convey my sincere thanks to our Principal **Dr. M. Davamani Christoher, M.Sc., M.Phil., M.Ed., PGDCA, Ph.D.(Inter Disciplinary), Ph.D.(Math).**

I am Grateful to the Head of our department **Mrs. T. Suganya, M.Sc., M.Phil., SLET, (Ph.D)** for her energetic encouragement and support.

I express my deep sense of gratitude and indebtedness to **Dr. S. Shaik Parveen, M.Sc., B.Ed., M.Phil., Ph.D.,** her inspiring guidance and continuous encouragement which has enabled me to complete this project.

Also I would like to thank my Teachers, Friends and my well wishers who have helped me during the course of completion of this project.

CONTENTS

S. NO.	DESCRIPTION	PAGE NO.
	Abstract	1
1	System Analysis 1.1 Existing System 1.2 Proposed System 1.3 Feasibility Study	2 2 3
2	System Requirements Specification 2.1 Software Requirements 2.2 Hardware Requirements 2.3 About Software	6 6 7
3	System Design 3.1 Modules Description 3.2 Functional Requirements 3.2.1 System Diagram 3.2.2 Data Flow Diagrams 3.2.3 E-R Diagram 3.3 User Interface Design	12 15 16 18 19
4	System Implementation	20
5	System Testing	21
6	System Maintenance	23
7	Conclusion	25
8	Future Enhancement	26
9	Bibliography	27
10	Appendix 10.1 Sample Screens 10.2 Source Code	28 32

ABSTRACT

This Project focuses on the development of the Application that integrates Optical Character Recognition (OCR) and Text-to-Speech (TTS) Generation, that converts printed text to audible speech. The system employs OCR to capture text from Image, documents and Real-world Objects, Using Java's Machine Learning Technology, it accurately recognizes character and converts into machine-readable text.

The OCR-based Text Recognition and Speech Conversion System is an Android application designed to enhance accessibility by integrating Optical Character Recognition (OCR) and Text-to-Speech (TTS) functionalities. The application utilizes Google ML Kit for real-time text recognition from images and employs basic MLKit Enhancer for pre-processing, including grayscale conversion to improve OCR accuracy. The system allows users to capture an image using their device's camera, process the image to extract text, and then convert the recognized text into speech using TTS technology.

The app follows a streamlined workflow: upon capturing an image, the application automatically processes it, extracts the text, and navigates to a display screen where users can review the text. Users can then opt to listen to the extracted text via a built-in speech synthesis module. The application is designed with an intuitive user interface and ensures smooth processing for real-time interactions.

This project is particularly useful for visually impaired individuals, aiding them in accessing printed text in a more convenient manner. It also has applications in language learning, document digitization, and accessibility enhancement. By leveraging machine learning and computer vision, this system provides an efficient and user-friendly approach to text recognition and speech conversion on mobile devices.

CHAPTER 1

SYSTEM ANALYSIS

1.1. EXISTING SYSTEM

OCR Systems

It supports multiple languages and provides high accuracy for text recognition, scanned in documents.

TTS Systems

Provides natural-sounding speech and supports a variety of languages and voices. The most popular OCR and TTS system we know is GOOGLE LENS which runs via Internet access.

Performance Issues

OpenCV and Tesseract OCR are resource-intensive, leading to slow processing speeds on low-end devices. Real-time text detection is not always accurate, especially in noisy or low-light conditions.

No Built-in Text-to-Speech Support

The existing systems do not provide native Text-to-Speech (TTS) conversion. Users need to integrate separate TTS libraries manually .

1.2. PROPOSED SYSTEM

Integrated OCR and TTS System

Develop a unified platform that extracts text from images and converts it into natural sounding speech.

Enhanced Accessibility

Focus on creating tools to assist visually impaired users by reading out text from documents and images. My Application can Capture and Convert to speech without internet access.

Real-Time Text Recognition using Google ML Kit

Replaces OpenCV and Tesseract OCR with Google ML Kit for faster and more efficient text detection. Works without requiring external dependencies like OpenCV's native C++ support (NDK). Accurately detects text from images captured using the device's built-in camera.

Seamless Camera Integration

Uses Android's CameraX API for smooth camera preview and image capturing. Eliminates the need for manually processing the image before text extraction.

Offline Functionality

Works without an internet connection as ML Kit's on-device text recognition model is used. Ensures low latency and fast processing compared to cloud-based OCR solutions.

1.3. FEASIBILITY STUDY

Technical Feasibility

The Optical Character Recognition (OCR) and Text-to-Speech (TTS) Generation project is technically feasible due to the availability of advanced tools, libraries, and frameworks. The implementation relies on Google ML Kit for text recognition, which provides efficient and accurate OCR capabilities. The Text-to-Speech (TTS) API ensures smooth and natural speech synthesis, making the extracted text more accessible to users.

Programming Language - Java.

Frameworks & Libraries - AndroidX, Google ML Kit (Text Recognition API), CameraX, TTS (Text-to-Speech)

Tools - Android Studio, Gradle, Firebase (if needed for future expansion).

Economic Feasibility

The Optical Character Recognition (OCR) and Text-to-Speech (TTS) Generation project is economically feasible due to its cost-effective implementation. It primarily utilizes open-source tools and libraries, such as Google ML Kit for text recognition and Android's built-in Text-to-Speech (TTS) API, eliminating the need for expensive third-party software or hardware.

The development environment, Android Studio, is free to use, reducing initial costs. The project requires only a smartphone camera for image capture, eliminating the need for additional hardware investments.

Development costs : Low (open-source tools, no expensive third-party APIs)

Deployment costs : Minimal (Google Play Store fee if published)

Maintenance costs : Low, as ML Kit updates are managed by Google.

Operational Feasibility

The Optical Character Recognition (OCR) and Text-to-Speech (TTS) Generation project is highly operationally feasible as it is designed to be user-friendly, efficient, and accessible. The application runs on Android devices, making it widely usable without requiring specialized hardware.

The workflow is streamlined: users capture an image using their phone camera, the system processes the image, extracts text using Google ML Kit, and then converts it to speech using Android's Text-to-Speech (TTS) API. The app operates in real-time, ensuring a smooth user experience.

User Benefits

Easy-to-use interface with a simple capture and detect workflow. Works offline, ensuring accessibility even in low-network areas. Suitable for students, professionals, and visually impaired users

System Usability

Uses CameraX for seamless image capturing. Displays text instantly after detection. Built-in Text-to-Speech (TTS) for accessibility.

Legal & Compliance Feasibility

Intellectual Property Rights – The project must use licensed or open-source tools such as Google MLKit for OCR and Android's Text-to-Speech API, ensuring that no proprietary technology is used without proper authorization.

Accessibility Compliance – Since the project includes TTS functionality, it should follow accessibility guidelines like the WCAG (Web Content Accessibility Guidelines) to ensure usability for visually impaired individuals.

Terms of Service Compliance (If Published) – The app must adhere to the Google Play Store policies if published, ensuring no violation of content, permissions, or data collection rules.

CHAPTER 2

SYSTEM REQUIREMENTS SPECIFICATIONS

2.1. SOFTWARE REQUIREMENTS

Operating System: Windows 10/11, macOS, or Linux (Ubuntu recommended)

Development Tools & IDE

Android Studio (Latest version)	- For coding, debugging, and testing
Gradle management	- For build automation and dependency
Emulator or Physical Device	- For testing (Pixel 5, Pixel 6, or any Android phone)

Programming Language & Frameworks

Java/Kotlin	- Primary language for Android development
Android SDK & AndroidX Libraries	- For core Android functionalities
ML Kit (Google Play Services)	- For on-device text recognition
CameraX API	- For camera handling and capturing images

Libraries & Dependencies

“com.google.mlkit:text-recognition:16.0.0” - ML Kit for text detection.

“androidx.camera:camera-core:1.2.3” - CameraX for image processing.

2.2. HARDWARE REQUIREMENTS

For Development (PC/Laptop)

Processor:	Intel i3/i5 or AMD Ryzen 5 (or higher)
RAM:	Minimum 8GB (Recommended 16GB for smooth performance)
Storage:	Minimum 256GB SSD (Recommended 512GB SSD for faster builds)
Graphics:	Integrated GPU like Intel Iris XE or UHD 620/730/770 graphics (Dedicated GPU recommended for better emulator performance)
USB Ports:	For connecting physical Android devices for debugging

For Android Device (Testing & Deployment)

Android Version:	Minimum Android 7.0 (API 24), Recommended Android 10+ (API 29+)
Processor:	Snapdragon 4 Gen 2 or higher (for real-time processing)
RAM:	At least 3GB RAM (for smooth camera + text recognition processing)
Camera:	Minimum 8MP (for high-quality image capture and better text detection)

2.3. ABOUT SOFTWARE

Introduction to Android Studio

Android Studio is the official Integrated Development Environment (IDE) for Android app development, developed by Google and based on IntelliJ IDEA by JetBrains. It provides a powerful and user-friendly environment for building Android applications efficiently. Android Studio supports Java, Kotlin, and C++ as primary programming languages and offers numerous tools to streamline the development process.

Key Features of Android Studio

Intelligent Code Editor – Android Studio comes with a powerful code editor that provides features like code completion, syntax highlighting, and real-time error detection, making it easier for developers to write and debug code.

Layout Editor – The drag-and-drop UI designer helps developers create user interfaces visually. It allows real-time preview of how the app will look on different screen sizes.

Gradle Build System – Android Studio uses Gradle to automate the build process, allowing developers to manage dependencies and create different app versions efficiently.

Emulator – The built-in Android Emulator lets developers test apps on a variety of virtual devices without needing physical hardware.

APK Analyzer – Helps in inspecting APK files to reduce size and optimize app performance.

Logcat & Debugger – Provides real-time logging and debugging tools to detect errors and analyze app behavior.

Support for Multiple Platforms – Developers can build apps for smartphones, tablets, smart TVs, Android Wear (smartwatches), and Android Auto.

Performance Profilers – Allows monitoring CPU, memory, and network usage to optimize app performance.

Introduction to Java

Java is a high-level, object-oriented, and platform-independent programming language developed by James Gosling at Sun Microsystems in 1995. It is widely used for developing web applications, mobile apps, desktop software, and enterprise solutions. Java follows the "Write Once, Run Anywhere" (WORA) principle, meaning that Java programs can run on any platform with a Java Virtual Machine (JVM).

Explanation to Java

Java is an object-oriented programming (OOP) language, meaning it is based on the concept of classes and objects. It supports key OOP principles such as inheritance, polymorphism, encapsulation, and abstraction. Java is also secure, as it runs in a controlled environment within the JVM, reducing system vulnerabilities.

Java programs are compiled into bytecode, which is then executed by the JVM. This makes Java highly portable across different operating systems like Windows, macOS, and Linux.

It also provides features such as automatic memory management (Garbage Collection), which helps in efficient resource utilization.

Java is used in various fields, including Android development, web development (Spring, Hibernate), cloud computing, and big data. Its simplicity, reliability, and extensive libraries make Java one of the most popular programming languages in the world.

Introduction to Google ML Kit

Google ML Kit is a powerful machine learning (ML) development toolkit provided by Google for Android and iOS applications. It enables developers to integrate on-device ML capabilities easily without requiring deep knowledge of machine learning. ML Kit provides ready-to-use APIs for tasks like text recognition, face detection, barcode scanning, object detection, and more. Since ML Kit runs on-device, it ensures fast processing, offline support, and better user privacy.

Explanation of Google ML Kit

Google ML Kit simplifies machine learning by offering pre-trained models that can be integrated into mobile apps with minimal coding. Developers can access these models through APIs such as Text Recognition, Image Labeling, Pose Detection, Smart Reply, and Language Identification. These APIs help applications recognize and process text, detect faces, and even analyze emotions.

One of ML Kit's major advantages is its ability to run on-device, reducing reliance on cloud computing. This leads to faster response times, improved security, and reduced data costs. Additionally, ML Kit supports custom model integration, allowing developers to use TensorFlow Lite models for specific use cases.

With ML Kit, developers can create intelligent, AI-driven mobile applications that enhance user experience through real-time, efficient, and reliable machine learning capabilities.

Introduction to XML

XML (Extensible Markup Language) is a widely used language for structuring and storing data in a human-readable and machine-readable format. It was designed to be both flexible and platform-independent, making it an essential tool for data representation and transfer across different systems.

Explanation to XML

XML uses tags to define and organize data in a structured manner, similar to HTML. However, unlike HTML, XML does not have predefined tags; users can create custom tags based on their needs. XML documents consist of a hierarchical structure with a root element, child elements, attributes, and text data.

In Android development, XML is used extensively for designing UI layouts, defining configuration settings, and structuring data interchange between applications. For example, Android UI layouts, defined in XML files, help separate the presentation layer from the application's logic.

XML is also widely used in web services, RSS feeds, SOAP-based APIs, and configuration files. Since XML is both human-readable and machine-readable, it remains a preferred format for structured data storage and exchange across platforms. While JSON has become a popular alternative, XML is still widely used in many modern applications due to its flexibility and extensibility.

Introduction to Gradle

Gradle is a powerful build automation tool used primarily for Java, Android, and other JVM-based projects. It is designed to automate the process of compiling, testing, packaging, and deploying applications efficiently. Gradle is known for its flexibility, scalability, and high performance, making it a preferred choice for Android development and enterprise-level projects.

Explanation to Gradle

Gradle follows a declarative build script approach using Groovy or Kotlin to define build configurations. It provides dependency management, incremental builds, and parallel execution, ensuring faster and more efficient build processes.

Key Features of Gradle

Dependency Management – Gradle integrates with repositories like JCenter to fetch required libraries automatically.

Incremental Builds – It only recompiles files that have changed, reducing build time significantly.

Task Automation – Developers can define custom tasks, automating repetitive actions like code generation and testing.

Multi-Project Support – Gradle supports large projects with multiple modules and dependencies.

Plugin System – Various plugins, including the Android Gradle Plugin, extend functionality and simplify project configurations.

Gradle in Android Studio

Android Studio uses Gradle as its default build system. The `build.gradle` file (written in Groovy/Kotlin) specifies dependencies, compile options, and project configurations. Gradle syncs the project, ensuring all required libraries and modules are correctly integrated.

With its modern, efficient, and customizable architecture, Gradle plays a crucial role in software development, providing developers with a robust and scalable build system.

CHAPTER 3

SYSTEM DESIGN

3.1. MODULE DESCRIPTION

User Interface Module

This is the front windows of the Application where the user choose the options like, Capture, and Go to Speak Activity. User get in with this module and asked to use the options that are represented here. The User Interface (UI) module of the Optical Character Recognition (OCR) and Text-to-Speech (TTS) Generation project is designed to provide an intuitive and user-friendly experience. It includes key elements such as a camera preview screen for capturing images, a text display area for recognized text, and a TTS control panel for converting detected text into speech. The UI ensures smooth navigation between different screens, allowing users to capture images, view extracted text, and listen to the converted speech. Implemented using XML layouts and Android UI components, the module enhances accessibility and user interaction, ensuring a seamless experience

Image Acquisition Module

The Image Acquisition Module in the Optical Character Recognition (OCR) and Text-to-Speech (TTS) Generation project is responsible for capturing images in real-time using the device's camera. This module allows users to take a picture of printed or handwritten text, which is then processed for text recognition. It utilizes the CameraX API for efficient image handling and ensures high-quality image capture for accurate text extraction.

Upon clicking the capture button, the module opens the camera, lets the user take a picture, and then automatically passes the image to the image processing module. The captured image is converted to grayscale to improve contrast before applying OCR for text detection. Additionally, the module ensures proper camera focus, exposure, and lighting adjustments for enhanced readability. This implementation provides a smooth, user-friendly experience and is essential for the accuracy of the OCR system in extracting text effectively.

Technologies Used: Android CameraX API.

Preprocessing Module

The Preprocessing Module is a crucial step in the Optical Character Recognition (OCR) and Text-to-Speech (TTS) Generation system. It enhances the quality of the captured image to improve text recognition accuracy. After capturing an image, this module applies grayscale conversion, noise reduction, binarization, and contrast enhancement to make the text clearer.

Grayscale conversion removes color information, making text processing more efficient. Thresholding techniques, such as Otsu's Binarization, help distinguish text from the background. Denoising filters remove unwanted distortions, ensuring clean edges around the text. The processed image is then passed to the OCR engine (Google ML Kit) for accurate text extraction. This module is essential for optimizing text clarity, improving recognition accuracy, and ensuring reliable TTS conversion.

Technologies Used: Google ML Kit.

Text extraction Module

The Text Extraction Module is a key component of the Optical Character Recognition (OCR) and Text-to-Speech (TTS) Generation system. It is responsible for identifying and extracting text from the preprocessed image using Google ML Kit's Text Recognition API.

Once the image is captured and processed (grayscale conversion, noise reduction, and binarization), the OCR engine analyzes the image, detects characters, and converts them into machine-readable text. Google ML Kit's on-device OCR efficiently recognizes printed and handwritten text in multiple languages. It segments the text into blocks, lines, and words, ensuring structured text extraction.

The extracted text is displayed on the screen for verification. Users can edit or modify the text before proceeding with Text-to-Speech (TTS) conversion. This module ensures high accuracy by recognizing different fonts, styles, and orientations, making it suitable for scanned documents, books, signboards, and handwritten notes.

Technologies Used: Google ML Kit – Text Recognition API.

Voice Wave Module

The Voice wave Module in the Optical Character Recognition (OCR) and Text-to-Speech (TTS) Generation system is responsible for converting extracted text into natural-sounding speech. This module utilizes Google's Text-to-Speech (TTS) API, which processes the recognized text and synthesizes human-like audio output.

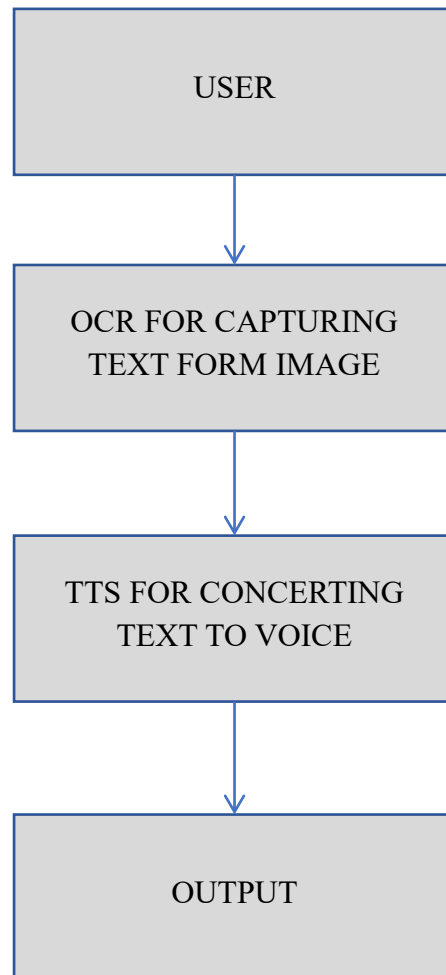
Once the text is extracted using OCR, the user can select a voice option (such as male or female) and adjust speech parameters like pitch and speed. The module then converts the text into speech and plays it aloud, enabling users to listen to the extracted content.

This module enhances accessibility for visually impaired individuals, students, and professionals by allowing them to listen to printed text in an audible format. By integrating TTS technology, the system ensures a seamless user experience for reading and understanding textual information.

Technologies Used: Google Text-to-Speech API.

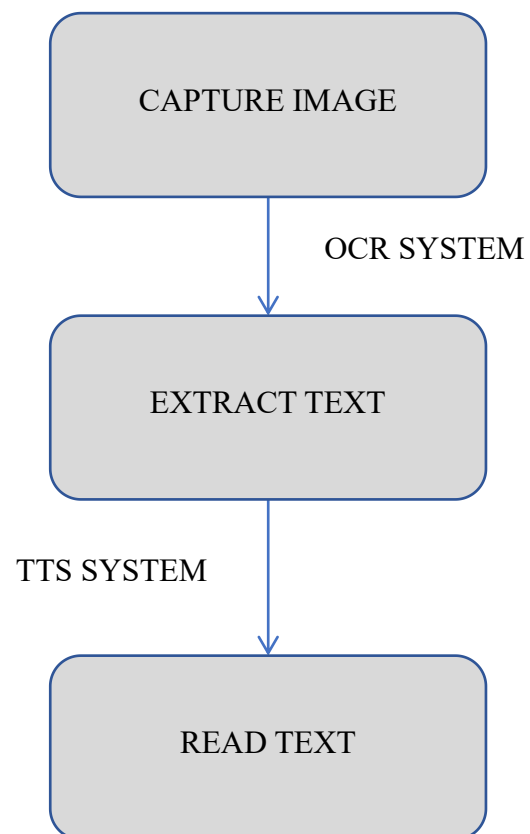
3.2. FUNCTIONAL REQUIREMENTS

3.2.1. SYSTEM DIAGRAM

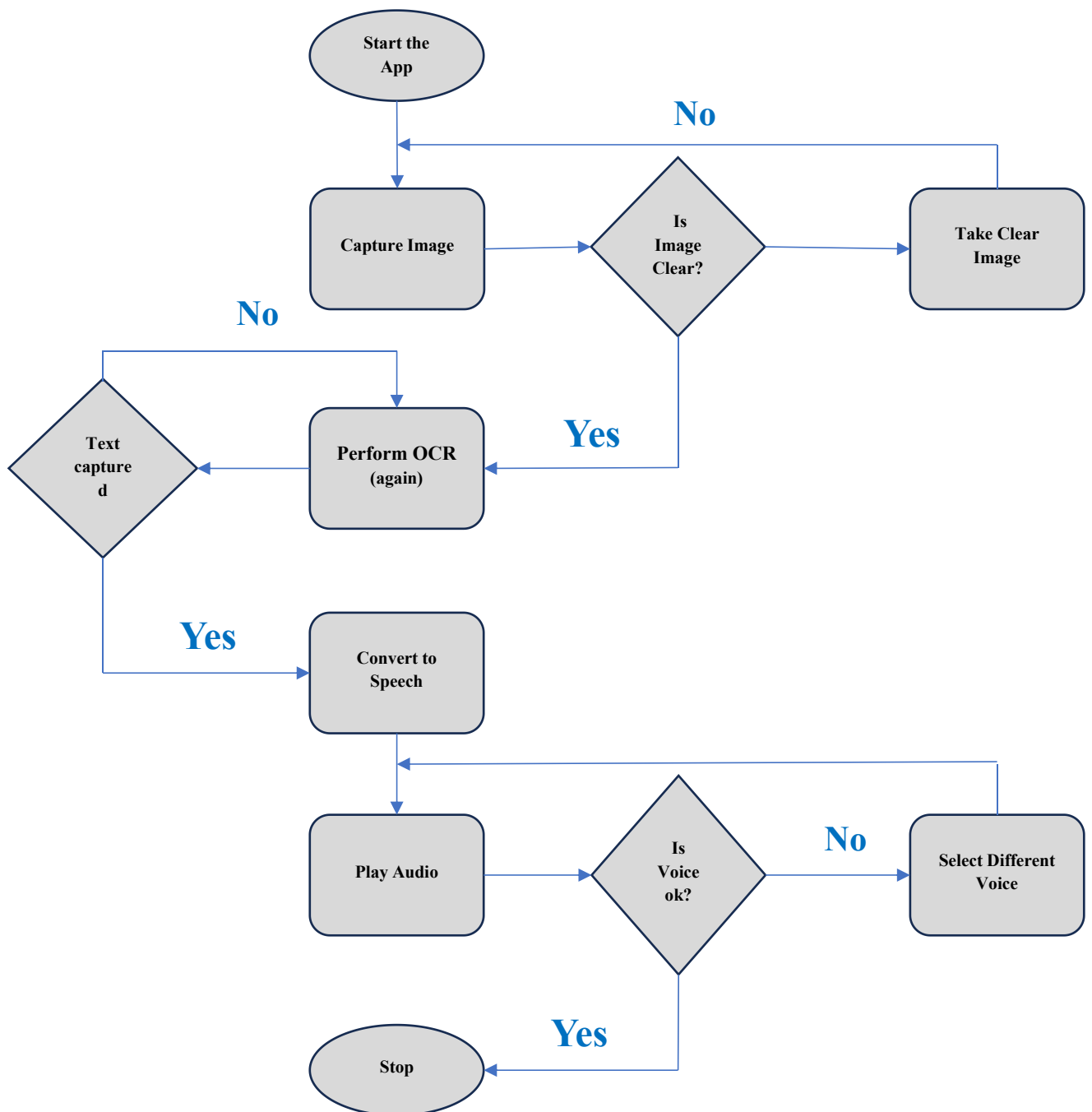


3.2.2. DATA FLOW DIAGRAM

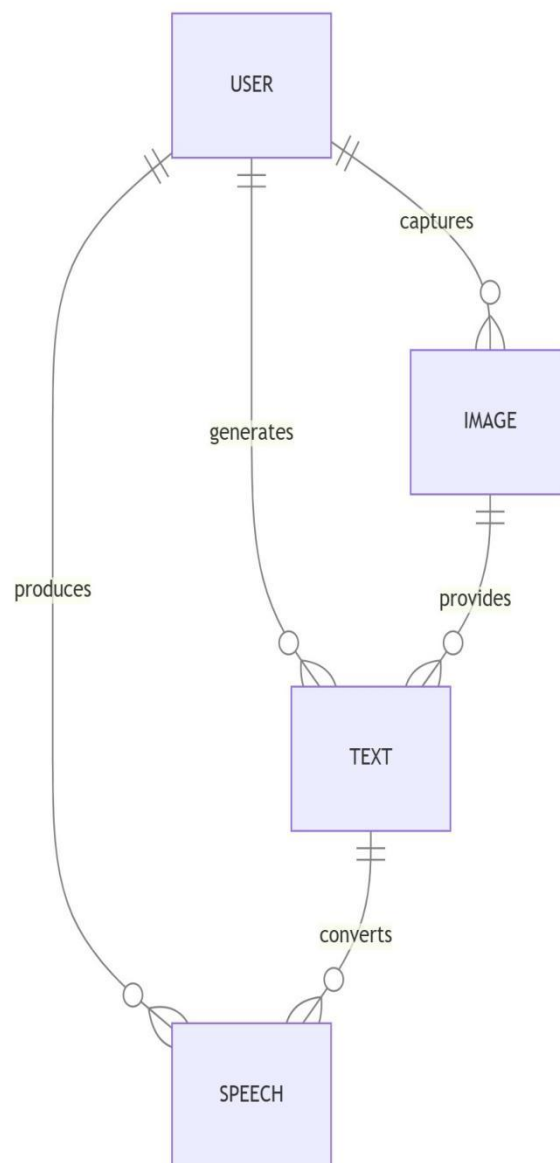
Level 0



Level 1



3.2.3. ER - DIAGRAM



3.3. USER INTERFACE DESIGN

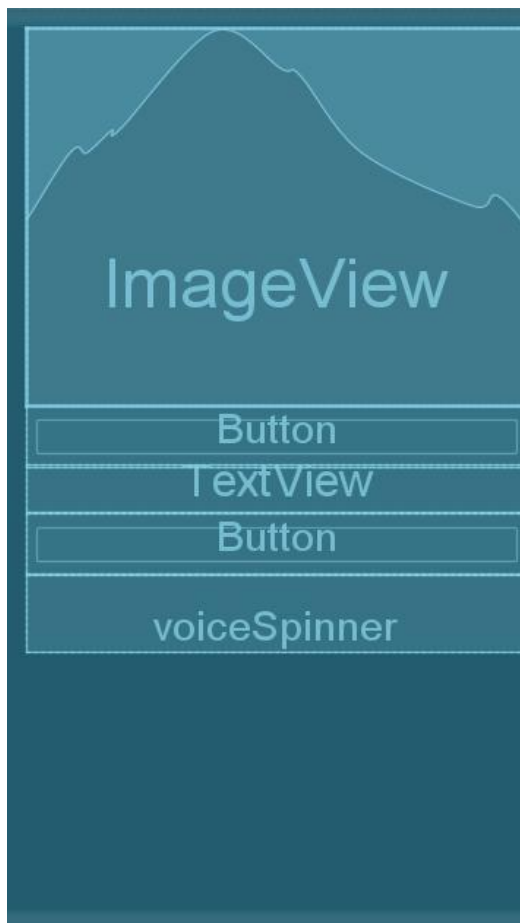


Image 1

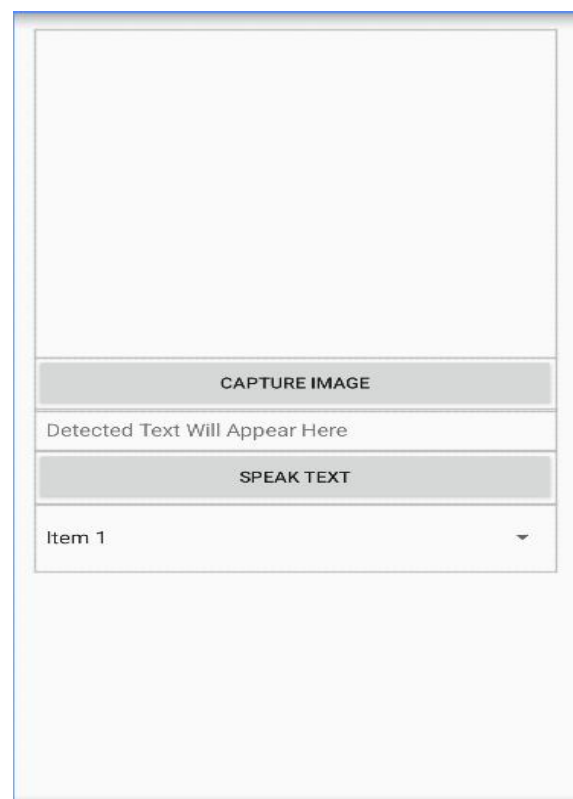


Image 2

CHAPTER 4

SYSTEM IMPLEMENTATION

Setting Up the Android Project

Open Android Studio and create a new Android project.

Set Minimum SDK to API Level 29 (Android 10.0) or higher.

Use Java as the development language.

Adding Necessary Dependencies

Open the “build.gradle (Module:app)” file to include the required dependencies.

Now, Sync the gradle with the Project to download all libraries.

Requesting camera Permission

In “AndroidManifest.xml”, add the necessary permissions.

Designing the UI (XML Layout)

Create “activity_main.xml” for main Interface.

Then Create “activity_speak.xml” for speak activity Interface.

Implementing Camera and Image Capture

Modify “MainActivity.java” to Handle camera preview and Image capture.

Running the App

Build and run the app on a real device or emulator.

Grant camera permission when prompted.

Capture an image, and the app will extract and display text from the image.

CHAPTER 5

SYSTEM TESTING

Unit Testing

Objective: To verify the functionality of individual components like camera preview, image capture, and text recognition.

Test Cases for Unit Testing

Test Case ID	Test Scenario	Expected Output	Status
UT-01	Verify camera initialization	Camera should start without errors	Pass
UT-02	Capture image successfully	Image should be processed without crashing	Pass
UT-03	Process text using ML Kit	Recognized text should appear on the screen	Pass

Integration Testing

Objective: To test the interaction between different components like CameraX, ML Kit, and UI elements.

Test Cases for Integration Testing

Test Case ID	Test Scenario	Expected Output	Status
IT-01	Open camera and display preview	Camera preview should be visible	Pass
IT-02	Capture button should trigger image processing	ML Kit should extract text from the image	Pass
IT-03	Display recognized text in UI	Extracted text should be shown in “TextView ”	Pass

Functional Testing

Objective: To check whether the app meets all functional requirements.

Test Cases for Functional Testing

Test Case ID	Functionality	Expected Result	Status
FT-01	App should request camera permission	Permission dialog should appear	Pass
FT-02	App should detect and extract text from images	Extracted text should be accurate	Pass
FT-03	The UI should update after text extraction	“ TextView ” should display extracted text	Pass

Performance Testing

Objective: To test the speed and efficiency of text recognition and camera operations.

Performance Metrics:

Camera Preview Start Time → < 2 seconds

Image Processing Time → < 3 seconds

Memory Usage → Optimized to prevent crashes.

CHAPTER 6

SYSTEM MAINTENANCE

Corrective Maintenance (Bug Fixing)

Objective

Identify and fix issues reported by users or found in testing.

Activities

Fixing crashes related to camera preview and image capture.

Resolving incorrect text detection by optimizing ML Kit settings.

Addressing UI responsiveness issues on different screen sizes.

Adaptive Maintenance (Updating for New Environments)

Objective

Keep the app working with new Android versions and devices.

Activities

Ensuring compatibility with latest Android SDK (e.g., Android 15+).

Updating dependencies (ML Kit, CameraX, etc.) to avoid outdated libraries.

Supporting different device screen resolutions and aspect ratios.

Perfective Maintenance (Performance & Feature Enhancements)

Objective

Improve speed, accuracy, and usability based on user feedback.

Activities

Enhancing text detection for blurry images using pre-processing techniques.

Adding support for multiple languages in text recognition.

Optimizing battery and memory usage to prevent overheating and lagging.

Preventive Maintenance (Security & Reliability)

Objective

Ensure the app remains secure and stable.

Activities

Regular security updates to protect against vulnerabilities.

Implementing data encryption to secure captured text.

Code refactoring to remove unused or inefficient code.

CHAPTER 7

CONCLUSION

The Optical Character Recognition (OCR) and Text-to-Speech (TTS) Generation project successfully combines image processing, text extraction, and speech synthesis to provide a seamless user experience. By capturing an image of printed text, the system extracts characters using Google ML Kit's OCR technology and converts them into speech through Google's Text-to-Speech API.

This project enhances accessibility by enabling visually impaired individuals, students, and professionals to listen to printed text effortlessly. The integration of real-time image processing and voice synthesis ensures a smooth workflow, improving both usability and efficiency.

With further advancements, this system can support multiple languages, offline processing, and improved speech synthesis for better clarity. Overall, the project demonstrates a practical application of machine learning and artificial intelligence in daily life, making information more accessible and interactive for users of all backgrounds.

Key Features Implemented

Camera-based Text Recognition – Uses ML Kit OCR to extract text from images.

Image Enhancement – Improves text clarity with Google ML Kit filters (grayscale, contrast, sharpening).

Text-to-Speech Conversion – Reads aloud the extracted text using Google TTS.

User-Friendly Navigation – Captured text is displayed and can be spoken with a single tap.

CHAPTER 8

FUTURE ENHANCEMENT

Support for Multiple Languages – Extend OCR & TTS to support multiple languages.

Handwritten Text Recognition – Enhance OCR to detect handwritten text also.

Save Extracted Text – Implement local storage or cloud integration to copy or save extracted text.

UI and User Interface enhancement – Provide a dark mode option for better visibility at night and Customizable themes.

Improving image quality - Image enhancement using OpenCV libraries(Using high Graphical source for Improving Image enhance to capture the Text clearly).

CHAPTER 9

BIBLIOGRAPHY

Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)

By Kim, S., & Chung, J. (2020).

Geeks for Geeks. (n.d.). Implementing Google ML Kit for Text Recognition in Android.

Retrieved from: <https://www.geeksforgeeks.org>

Google ML Kit Documentation

Google Developers. (n.d.). Text Recognition API

Retrieved from: [tps://developers.google.com/ml-kit/vision/text-recognition](https://developers.google.com/ml-kit/vision/text-recognition)

OpenCV Documentation

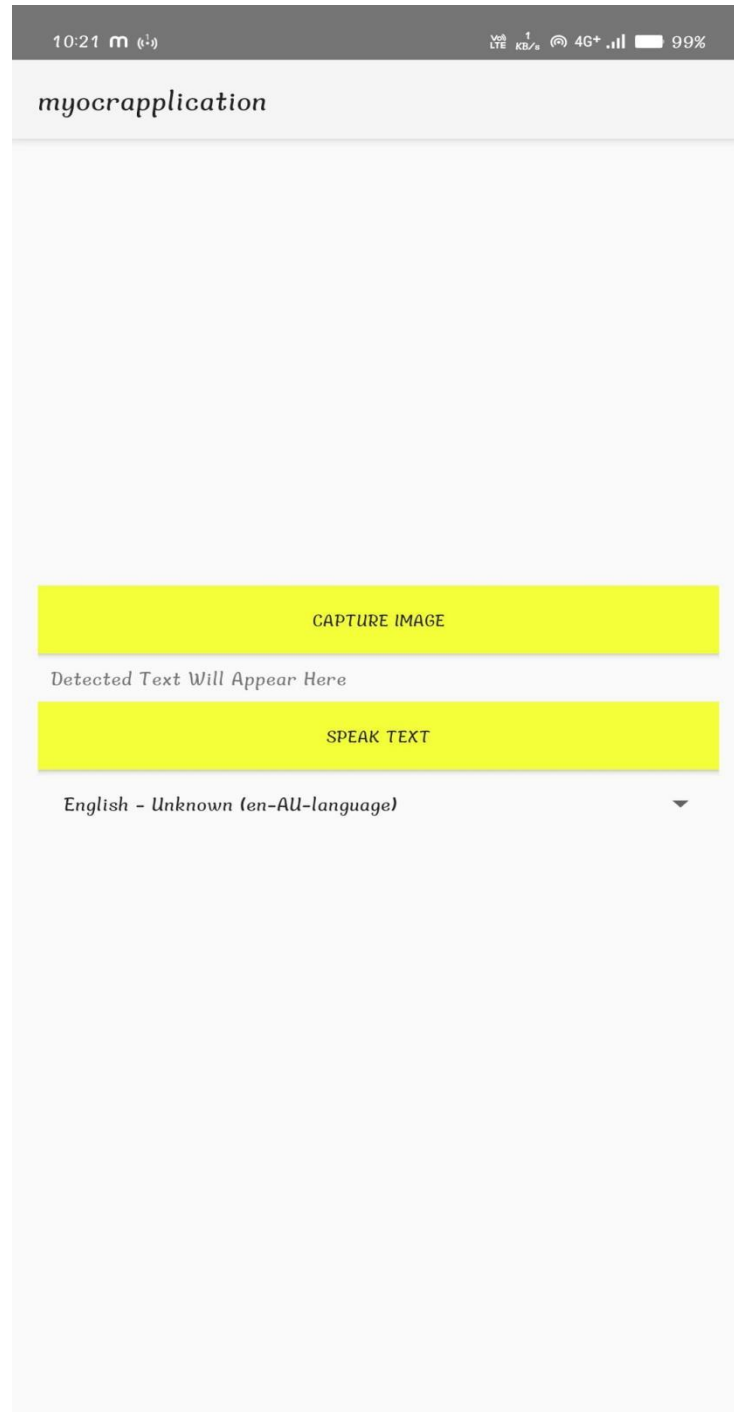
OpenCV. (n.d.). Image Processing Techniques in OpenCV

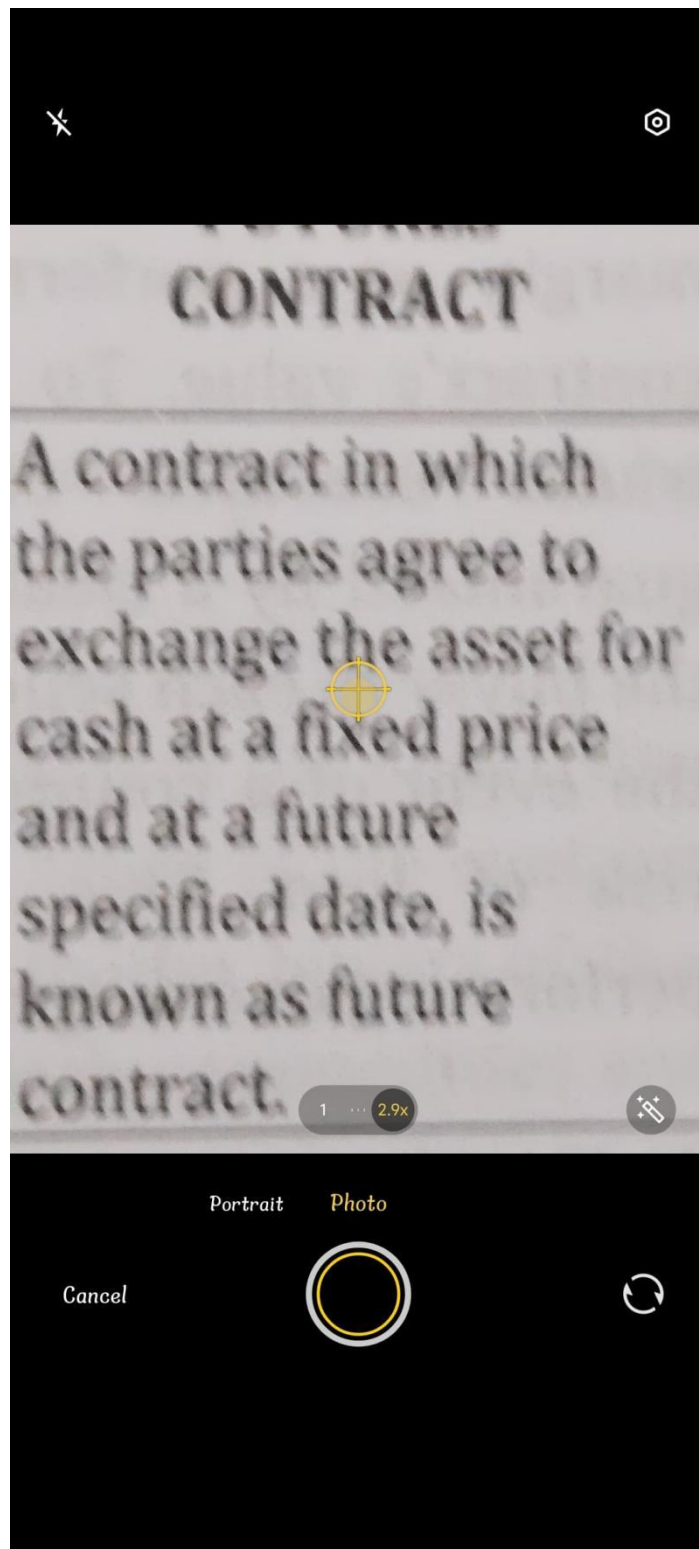
Retrieved from: <https://docs.opencv.org/4.x/>

CHAPTER 10

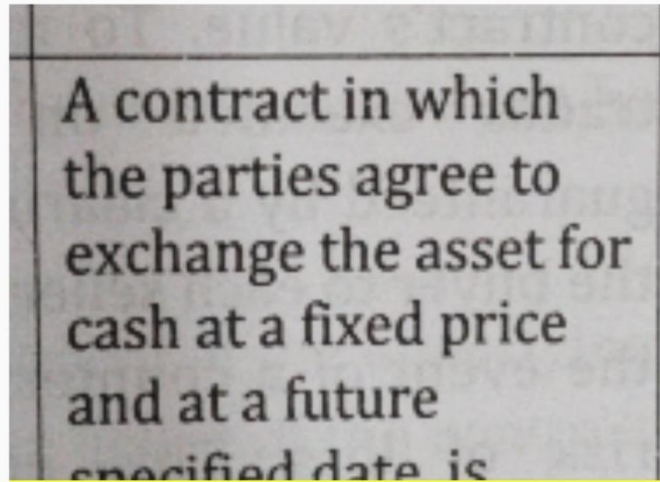
APPENDIX

10.1. SAMPLE SCREENS





myocrapplication



CAPTURE IMAGE

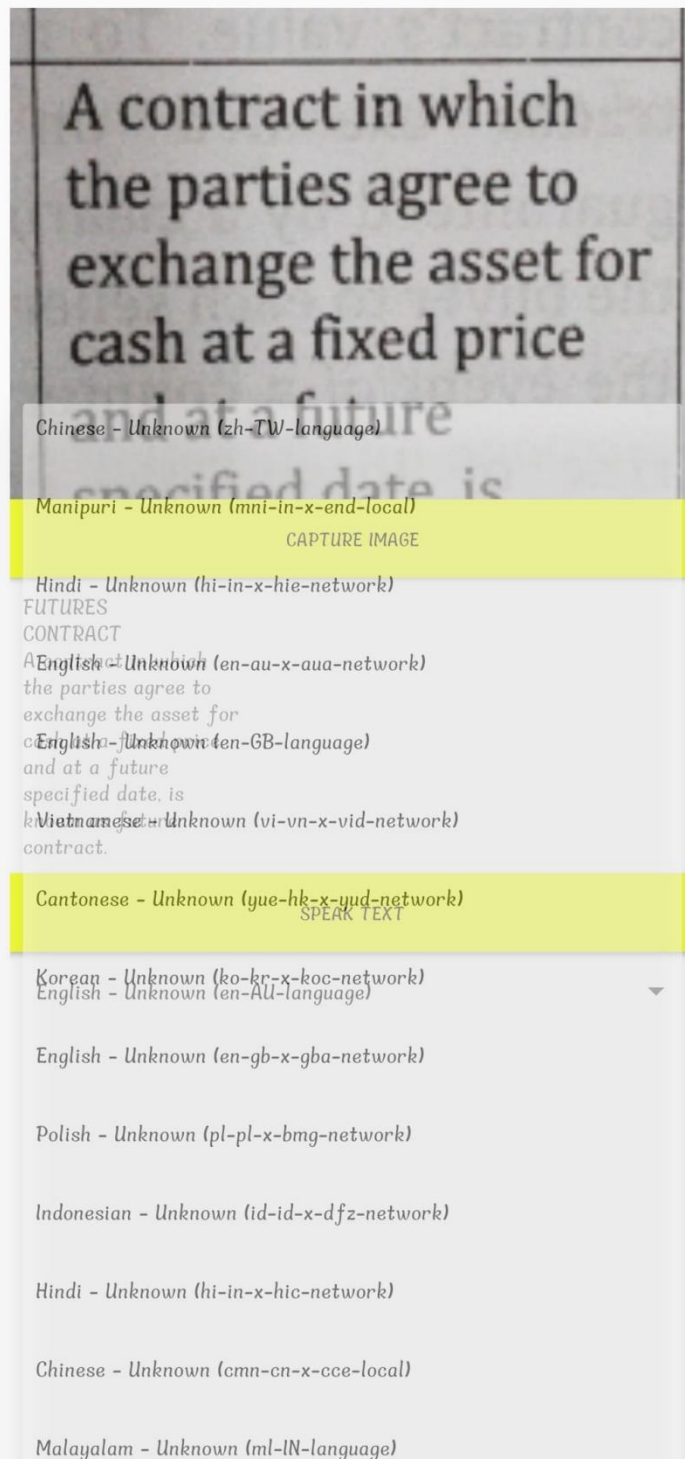
FUTURES
CONTRACT

A contract in which
the parties agree to
exchange the asset for
cash at a fixed price
and at a future
specified date, is
known as future
contract.

SPEAK TEXT

English - Unknown (en-AU-language) ▼

myocrapplication



10.2. SAMPLE SOURCE CODE

Build.Gradle.kts

```
plugins {  
    alias(libs.plugins.android.application)  
}  
android {  
    namespace = "com.example.myapplication"  
    compileSdk = 35  
  
    defaultConfig {  
        applicationId = "com.example.myapplication"  
        minSdk = 32  
        targetSdk = 34  
        versionCode = 1  
        versionName = "1.0"  
  
        testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"  
    }  
    buildTypes {  
        release {  
            isMinifyEnabled = false  
            proguardFiles(  
                getDefaultProguardFile("proguard-android-optimize.txt"),  
                "proguard-rules.pro"  
            )  
        }  
    }  
    compileOptions {  
        sourceCompatibility = JavaVersion.VERSION_11  
        targetCompatibility = JavaVersion.VERSION_11  
    }  
}  
dependencies {
```

```

implementation(libs.appcompat)
implementation(libs.material)
implementation(libs.activity)
implementation(libs.constraintlayout)
testImplementation(libs.junit)
androidTestImplementation(libs.ext.junit)
androidTestImplementation(libs.espresso.core)
implementation (libs.text.recognition)
}

```

Android_Manifest.xml

```

<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    package="com.example.myapplication">

    <uses-feature
        android:name="android.hardware.camera"
        android:required="false" />

    <uses-permission android:name="android.permission.CAMERA"/>
    <uses-permission android:name="android.permission.INTERNET"/>

    <application
        android:allowBackup="true"
        android:theme="@style/Theme.AppCompat.Light">

        <activity android:name=".MainActivity"
            android:exported="true"
            tools:ignore="MissingClass">
            <intent-filter>
                <action android:name="android.intent.action.MAIN"/>
                <category android:name="android.intent.category.LAUNCHER"/>
            </intent-filter>
        </activity>
    </application>

```

```
        </intent-filter>
    </activity>

</application>
</manifest>
```

MainActivity.java

```
package com.example.myapplication;

import android.content.Intent;
import android.graphics.Bitmap;
import android.os.Bundle;
import android.provider.MediaStore;
import android.speech.tts.TextToSpeech;
import android.speech.tts.Voice;
import android.util.Log;
import android.view.View;
import android.widget.*;
import androidx.annotation.Nullable;
import androidx.appcompat.app.AppCompatActivity;
import com.google.mlkit.vision.common.InputImage;
import com.google.mlkit.vision.text.Text;
import com.google.mlkit.vision.text.TextRecognition;
import com.google.mlkit.vision.text.TextRecognizer;
import com.google.mlkit.vision.text.latin.TextRecognizerOptions;
import java.util.Locale;
import java.util.Set;

public class MainActivity extends AppCompatActivity {

    private static final int REQUEST_IMAGE_CAPTURE = 1;
    private ImageView imageView;
    private TextView textView;
```

```

private Bitmap imageBitmap;
private TextToSpeech tts;
private Spinner voiceSpinner;
private String extractedText = "";

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);

    imageView = findViewById(R.id.imageView);
    textView = findViewById(R.id.textView);
    voiceSpinner = findViewById(R.id.voiceSpinner);
    Button btnCapture = findViewById(R.id.btnCapture);
    Button btnSpeak = findViewById(R.id.btnSpeak);

    // Initialize Text-to-Speech
    tts = new TextToSpeech(this, status -> {
        if (status == TextToSpeech.SUCCESS) {
            tts.setLanguage(Locale.US);
            loadVoices();
        }
    });

    btnCapture.setOnClickListener(v -> captureImage());
    btnSpeak.setOnClickListener(v -> speakText());
}

private void captureImage() {
    Intent takePictureIntent = new Intent(MediaStore.ACTION_IMAGE_CAPTURE);
    if (takePictureIntent.resolveActivity(getPackageManager()) != null) {
        startActivityForResult(takePictureIntent, REQUEST_IMAGE_CAPTURE);
    }
}

```



```

@Override
protected void onActivityResult(int requestCode, int resultCode, @Nullable Intent data) {
    super.onActivityResult(requestCode, resultCode, data);
    if (requestCode == REQUEST_IMAGE_CAPTURE && resultCode == RESULT_OK &&
data != null) {
        Bundle extras = data.getExtras();
        imageBitmap = (Bitmap) extras.get("data");
        imageView.setImageBitmap(imageBitmap);
        processImage();
    }
}

private void processImage() {
    if (imageBitmap != null) {
        InputImage image = InputImage.fromBitmap(imageBitmap, 0);
        TextRecognizer recognizer =
TextRecognition.getClient(TextRecognizerOptions.DEFAULT_OPTIONS);

        recognizer.process(image)
            .addOnSuccessListener(text -> {
                extractedText = text.getText();
                textView.setText(extractedText);
            })
            .addOnFailureListener(e -> Toast.makeText(this, "Failed to recognize text",
Toast.LENGTH_SHORT).show());
    }
}

private void loadVoices() {
    Set<Voice> voices = tts.getVoices();
    ArrayAdapter<String> adapter = new ArrayAdapter<>(this,
android.R.layout.simple_spinner_item);
    adapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);

    for (Voice voice : voices) {

```

```

        if (voice.getLocale().equals(Locale.US)) {
            adapter.add(voice.getName());
        }
    }

    voiceSpinner.setAdapter(adapter);
}

private void speakText() {
    if (!extractedText.isEmpty()) {
        String selectedVoice = (String) voiceSpinner.getSelectedItem();
        if (selectedVoice != null) {
            for (Voice voice : tts.getVoices()) {
                if (voice.getName().equals(selectedVoice)) {
                    tts.setVoice(voice);
                    break;
                }
            }
        }
        tts.speak(extractedText, TextToSpeech.QUEUE_FLUSH, null, null);
    } else {
        Toast.makeText(this, "No text to speak", Toast.LENGTH_SHORT).show();
    }
}

@Override
protected void onDestroy() {
    if (tts != null) {
        tts.stop();
        tts.shutdown();
    }
    super.onDestroy();
}
}

```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">

    <ImageView
        android:id="@+id/imageView"
        android:layout_width="match_parent"
        android:layout_height="300dp"
        android:scaleType="centerCrop"/>

    <Button
        android:id="@+id/btnCapture"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Capture Image"/>

    <TextView
        android:id="@+id/textView"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Detected Text Will Appear Here"
        android:textSize="16sp"
        android:padding="8dp"/>

    <Button
        android:id="@+id/btnSpeak"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Speak Text"/>
```

```

<Spinner
    android:id="@+id/voiceSpinner"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"/>

</LinearLayout>

```

Settings.gradle.kts

```

pluginManagement {
    repositories {
        google {
            content {
                includeGroupByRegex("com\\.\\.android.*")
                includeGroupByRegex("com\\.\\.google.*")
                includeGroupByRegex("androidx.*")
            }
        }
        mavenCentral()
        gradlePluginPortal()
    }
}

dependencyResolutionManagement {
    repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
    repositories {
        google()
        mavenCentral()
    }
}

rootProject.name = "My Application"
include(":app")

```